

Credo Payment Service API documentation

The Credo Payment Service API is a RESTful API built using the OpenAPI Standard (OAS). It is designed for developers to easily interact with Credo's payment services through predictable, resource-oriented URLs.

Key features:

- **Data Format:** The API accepts requests with JSON-encoded bodies and returns responses in JSON format.
- **Standard HTTP Methods:** It follows standard HTTP methods (GET, POST, PUT, DELETE), response codes, and error handling practices.
- **Authentication:** API authentication is done using API keys, ensuring secure access to Credo's services.

Base URLs:

- **Production (Live):** <https://api.credocentral.com>
- **Sandbox (Demo):** <https://api.credodemo.com>

Getting Started:

1. Sandbox Environment:

- To begin testing and integrating with Credo's API, create an account on our [sandbox environment](#)
- You can access your API key after signing up by visiting the [API key settings](#) page.

2. Live Environment:

- For production access, create an account on our [production environment](#).

- Complete the onboarding process to gain access to your production API key, which will be available **after approval** in the [API key settings](#) page.

3. Payment processing flow:

This flow outlines the **Merchant-Customer Interaction** during a payment process using the **Credo Payment Gateway (CPG)**.

- **Payment Initiation by Customer**

- The **Customer** initiates a payment through the **Merchant App**.
- This triggers a POST /transaction/initialize request to the **CPG**.

- **Transaction Initialization and Validation**

- Upon receiving the request, the **CPG** performs request validation.
- If validation fails, the **Merchant App** receives an HTTP 400 error with details of the failed validation, and the process halts.
- If validation succeeds, the **CPG** sends back an authorizationUrl, a unique transaction reference (transRef) and credo reference number (crn).

- **Redirecting the Customer to Authorization**

- The **Merchant App** redirects the **Merchant Customer** to the provided authorizationUrl for payment processing.
- The customer selects their payment channel on the **CPG** platform.

- **Payment Selection and Completion**

- The **Merchant Customer** completes the payment process on the **CPG**.
- This part is iterative and may involve multiple steps depending on the payment channel.

- **Payment Success and Redirection**

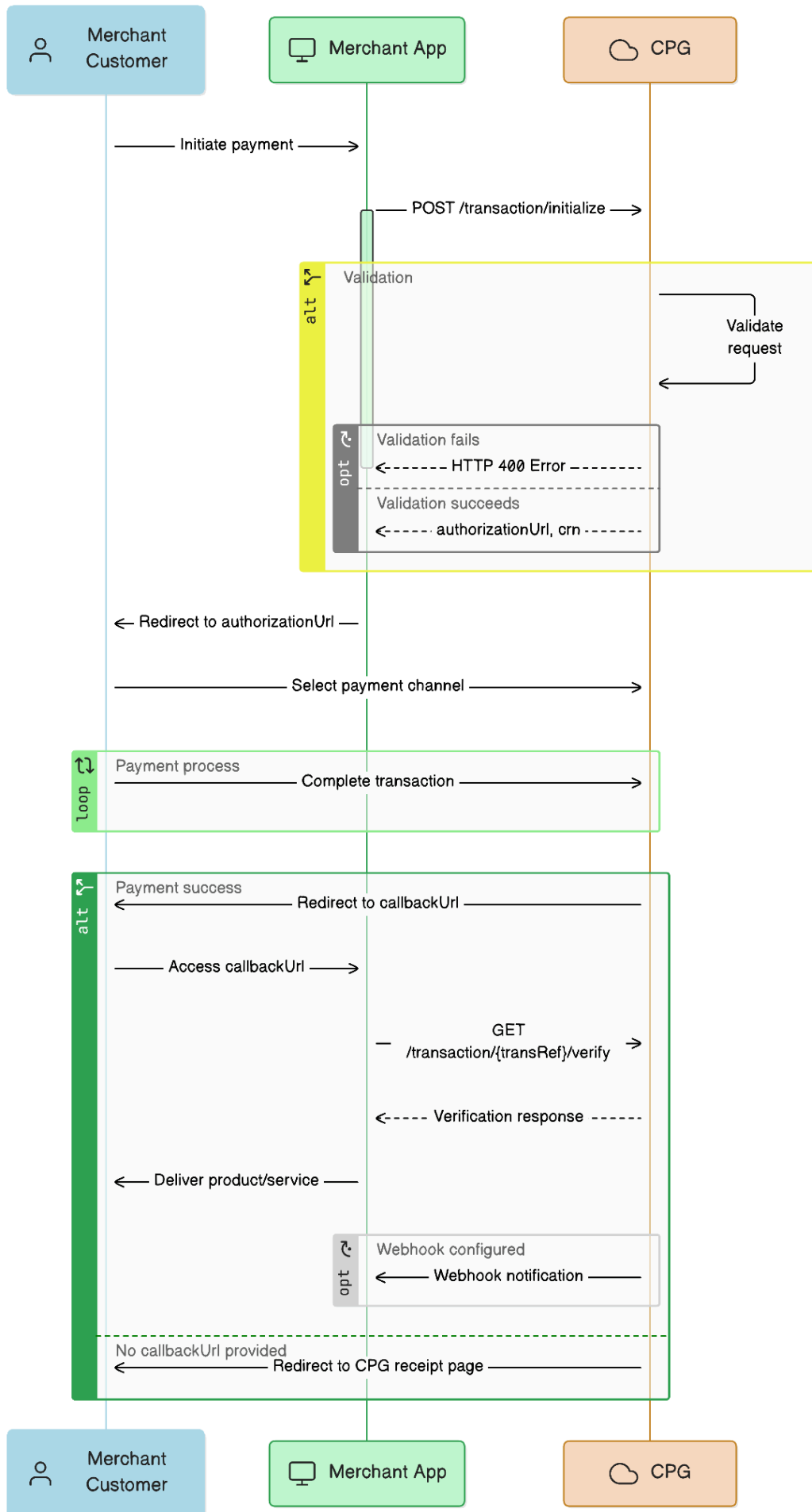
- On successful payment, the **CPG** redirects the **Merchant Customer** to the provided callbackUrl to notify the **Merchant App**.
- The **Merchant App** then performs a final transaction verification using the GET /transaction/{transRef}/verify endpoint.
- Once verified, the **Merchant App** delivers the purchased product or service to the **Merchant Customer**.

- **Webhook Notification (Optional)**

- If the **Merchant App** has configured a webhook, the **CPG** sends an additional notification via webhook for transaction updates.

- **No Callback URL**

- If no callbackUrl is provided during initialization, the **CPG** redirects the customer to a receipt page after completing the payment.



Payment Initialization

This endpoint allows you to initialize a payment request. Use this to initiate a transaction by providing the necessary payment details.

- **Path:** </transaction/initialize>
- **Request Type:** POST
- **Headers:**

Authorization: Provide your public API key here for authentication.

Example: Authorization: 0PUB0629zabkUVDdb5rpdM1nGo43F7yOy.

Content-Type: application/JSON

Specifies that the request body is JSON-encoded.

Param	Required	Description
api-public-key	Yes	Your public key . Use test key for test mode and live key for live mode
email	Yes	Email address of customer
amount	Yes	Amount (in the lowest currency value—kobo, pesewas, pesewas or cent) debiting the debiting customer.
reference	No	Unique case-sensitive transaction reference. Only alphanumeric characters are allowed. If you do not pass this parameter, Credo will generate a unique reference.
currency	No	Currency charge should be performed. Allowed values are NGN or USD. It defaults to your integration currency.
metadata	No	Object containing any extra information you want to be recorded with the transaction. Fields within the custom_field object will appear within the transaction information on the Credo Dashboard.
callbackUrl	No	After a payment attempt, we'll redirect the customer to this URL

serviceCode	No	Dynamic settlement service code
customerFirstName	No	Customers first name
customerLastName	No	Customers last name
customerPhoneNumber	No	Customers phone number
bearer	No	0 = Customer beares fee; 1 = Merchant beares fee
narration	No	Transaction narration
initializeAccount	No	Field 'initializeAccount' accepts 0=False(default) or 1=True; When you set 'initializeAccount' to true (1), it enables the gateway to generate a virtual bank account number for the transaction. Note: Webhook implementation is required.
pauseSettlement	No	This accepts 0=False(default) or 1=True; Enables merchants to temporarily halt the settlement process within the escrow system.
pauseSettlementDate	No	Defines the effective date for the settlement of a pause transaction. Format - <i>yyyy-MM-dd</i>

POST /transaction/initialize HTTP/1.1

Host: api.credodemo.com

Authorization: 0PUB0629zabkUVDb5rpdM1nGo43F7y0y

Content-Type: application/json

```
{
  "amount": 15000,
  "bearer": 1,
  "callbackUrl": "https://app.example.com/callback-url",
  "customerFirstName": "John",
  "customerLastName": "Wick",
  "customerPhoneNumber": "2348032132100",
  "email": "john.wick@credocentral.com",
  "metadata": {
    "customFields": [
      {"variable_name": "gender", "value": "Male", "display_name": "Gender"},
      {"variable_name": "dept", "value": "Agric", "display_name": "Department"}
    ]
  }
}
```

Note: You should save the authorizationUrl for each customer/payment and reuse it if their payment is still pending when they return to your app. This will help prevent multiple open payments for the same customer, avoiding duplicate transactions.

```
{
  "status": 200,
  "message": "Successfully processed",
  "data": {
    "authorizationUrl": "https://pay.credodemo.com/Sfhx00BA4801Zk1T4",
    "reference": "201220045mG7mTL022zQ09",
    "credoReference": "Sfhx00BA4801Zk1T4",
    "Crn": "0000298483"
  },
  "execTime": 4.64,
  "error": []
}
```

Response Parameters:

Field	Description
status	The HTTP status code of the request (200 indicates success).
message	Describes the result of the operation (Successfully processed).
authorizationUrl	URL where the payer will be redirected to complete the payment.
reference	The reference generated for the bill on your platform.
credoReference	The unique reference generated by Credo for the transaction.
crn	The Credo Reference Number (CRN) assigned to the transaction. For bank branch payment through Payoutlet

account	Details of the dynamic virtual bank account created for the payment.
account.accountNumber	The bank account number generated for the transaction.
account.accountName	The name associated with the bank account.
account.bankName	The name of the bank associated with the account.
account.expiryDate	The date and time when the virtual bank account will expire. Note, funds received after the expiry date will be auto reversed
account.amount	The total amount due for the bill. Note, any amount that is not exactly this amount will be auto reversed
billNumber	The bill number associated with the transaction.
billInformation	Additional information about the bill from LIRS
billInformation.amountDue	The outstanding amount for the bill.
billInformation.payerName	Name of the payer or entity responsible for the bill.
billInformation.agencyName	Name of the agency responsible for the bill.
execTime	The time taken (in milliseconds) to process the request.
error	Any error message if the request fails (empty in this case).

Payment Verification

To verify the status of a previously initiated payment, make a GET request to the following endpoint.

- **Path:** `/transaction/{transRef}/verify`
- **Request Type:** GET
- **Headers:**

Authorization: `secret api key` Provide your API key here for authentication.

Example: Authorization: 0PUB0629zabkUVDdb5rpdM1nGo43F7yOy.

Content-Type: application/JSON

Specifies that the request body is JSON-encoded.

- **Request URL Parameters:**

transRef: The transaction reference (transRef) obtained from the payment initialization response.

```
GET /transaction/{transRef}/verify HTTP/1.1
Host: api.credodemo.com
Authorization: 0PUB0629zabkUVDdb5rpdM1nGo43F7yOy
Content-Type: application/json
```

```
{
  "status": 200,
  "message": "Successfully processed",
  "data": {
    "transRef": "JunW00GkHm01vo0N96pk",
    "businessRef": "201220045mG7mTL022zQ09",
    "debitedAmount": 15325.00,
    "transAmount": 15000.00,
    "transFeeAmount": 325.00,
    "settlementAmount": 15000.00,
    "customerId": "john.wick@credocentral.com",
    "transactionDate": "2023-01-04 21:59:10",
```

```

    "currencyCode": "NGN",
    "status": 0,
    "metadata": [
      {
        "insightTag": "gender",
        "insightTagValue": "Male",
        "insightTagDisplay": "Gender"
      }
    ],
    "execTime": 0.0,
    "error": []
  }

```

Response Parameters:

Param	Description
data.businessCode	Business unique identification number within our platform
data.businessRef	Unique case-sensitive transaction reference. Only alphanumeric characters are allowed. If you do not pass this parameter, Credo will generate a unique reference.
data.channelId	Credo's internal channel ID
data.currencyCode	Currency charge should be performed. Allowed values are NGN or USD It defaults to your integration currency.
data.customerId	Customer email during transaction initialization
data.debitedAmount	Total amount that was debited from the customer
data.metadata	Object containing any extra information you want to be recorded with the transaction. Fields within the custom_field object will appear on merchant receipts and within the transaction information on the Credo Dashboard.
data.serviceCode	Dynamic settlement service code
data.settlementAmount	Amount that will be settled to business
data.status	Transaction status
data.transAmount	Requested/actual transaction amount
data.transFeeAmount	Transaction fee
data.transRef	Credo internal Unique case-sensitive transaction reference

data.transactionDate	Transaction processed date
error	Validation error message if any
execTime	Time it took to process this request
message	Response message describing the status of the request
status	Response status of the request

Test Cards

To test the payment flow, redirect the user to the authorizationUrl provided in the response of the payment initialization request. Once redirected, use one of the following test cards to simulate the payment process.

Available Test Cards:

Card Number	Expiry	CVV	Comments
4012000033330026	09/25	556	Standard test card
5555555555554444	09/25	556	Do not fill the 3DS form

How to use:

1. **Redirect:** After initializing the payment, redirect the user to the authorizationUrl provided in the response.
2. **Test Card Details:** Use one of the test card numbers above during the payment process.
 - For **3DS** flows, follow the instructions provided in the comments.
 - In cases where an OTP is needed, use the standard procedure to simulate the payment process.

Note: These test cards are only available in the demo environment and will not work for live transactions.

Test Bank Transfer

To simulate a bank transfer transaction and test the settlement notification feature, make a POST request to the following endpoint.

- **Path:** </transaction/providus/settlement/notification>
- **Request Type:** POST
- **Headers:**
 - X-Auth-Signature:**
be09bee831cf262226b426e39bd1092af84dc63076d4174fac78a2261f9a3d6e59744983b8326b69cdf2963fe314dfc89635cfa37a40596508dd6eaa
b09402c7

Sample Request Body:

```
{
  "sessionId": "{{${randomUUID}}}",
  "accountNumber": "9926496832",
  "tranRemarks": "Customer/AT68_TRF2MPT26w0q01916528146/0378689",
  "transactionAmount": 17544.54,
  "settledAmount": 17494.54,
  "feeAmount": 0,
  "vatAmount": 0,
  "currency": "NGN",
  "settlementId": "{{${randomUUID}}}",
  "sourceAccountNumber": "1017432705",
  "sourceAccountName": "M B C COMPUTER AND ACCESSORIES",
  "sourceBankName": "MONIEPOINT MICROFINANCE BANK",
  "channelId": "1",
  "tranDateTime": "{{tranDateTime}}"
}
```

Key Points:

- **sessionId & settlementId:** Must be unique UUID
- **accountNumber:** Use the account number generated from the </transaction/initialize> endpoint.

- **transactionAmount:** This must match the amount returned with the account number from the initialization.
- **settledAmount:** This must also match the settled amount returned with the account number.
- **currency:** Should be set to NGN.
- **tranDateTime:** Use the current date and time in the format YYYY-MM-DDTHH:mm:ssZ.

Note: This is only available in the demo environment and will not work for live transactions.

Webhook Payment Notification

Upon the successful completion of a transaction, we will send a webhook notification to a URL provided by you. This notification will inform you of the transaction details for your records or further processing. [Webhook URL configuration settings](#)

- **Request Method:** POST
- **Headers:**

Content-Type: application/JSON

credo-signature: SHA256(merchant-token + transRef + reference)

Webhook Parameters:

```
{
  "event": "transaction.successful",
  "data": {
    "transref": "n3fg005xtg00q9qi02yn",
    "reference": "qv1aa1011q",
    "debitedamount": 100.00,
    "transamount": 100.00,
    "transfeeamount": 5.00,
    "settlementamount": 95.00,
    "customerid": "larrie4christ@gmail.com",
    "transactionDate": 1728662440000,
    "currencyCode": "NGN",
```

```
"status": 0,
"crn": "00002323",
"paymentMethod": "Card",
"narration": "Integrated 24 hour pricing structure",
"customer": {
  "customeremail": "john.wick@gmail.com",
  "firstName": "Hayden",
  "lastName": "Harber",
  "phoneNo": "783-561-4854"
},
"metadata": [
  {
    "insightTag": "function",
    "insightTagValue": "Integrated grid-enabled installation",
    "insightTagDisplay": "toolset"
  }
]
}
```

Handling Webhook Notifications:

- Ensure your server is configured to accept POST requests at the URL provided.
- Respond with a **200 OK** status code upon successfully receiving and processing the webhook to acknowledge it.
- If your server fails to acknowledge the webhook, retries may be attempted maximum of 5 times.

Transaction Statuses

Status Code	Status text	Status Description
15	Initialising	The merchant had just sent the payment request and the gateway had returned the payment URL.
14	Initialised	The payment URL was successfully loaded on the customer's browser.
13	Attempted	Customer has attempted to make a payment
12	Attempted	Account number generated, expecting credit notification from the bank
10	Cancelled	Merchant cancelled the payment
9	Cancelled	Customer cancelled the payment
7	Declined	The declination of a transaction that failed the fraud check.
6	Review	Flagged transaction that requires human review.
5	Settled	Successful transaction that has been settled.
4	Settle	Successful transaction that is being queued for settlement.
3	Failed	Failed transaction.
2	Refund	Successful transaction that is queued for a refund.
1	Refunded	Successful transaction that has been refunded.
0	Successful	Successful transaction

Note: Records with status 14 & 15 are not considered transactions yet and might not show in the transaction history.

Note: When a payment attempt fails, customers remain on the gateway, where we provide the reason for the failure and present alternative payment options. This approach maximizes the chances of a successful payment, ensuring our merchants achieve nearly a 100% success rate. This behavior is the default, but merchants can also request a redirect to the callback URL after a failed attempt, along with specifying the number of failed attempts before the redirection occurs.

INTERSERVER TRANSACTION (Direct Card Charge)

Direct card charge allows you to charge cards directly through a server to server implementation which requires you to first be PCI DSS compliant (This implies that you have a PCI DSS certificate and understand the risk of handling sensitive data). This is useful if your customers are predominantly credit/debit card users and you'd prefer they manage payments via your apps interface.

Interserver transactions are performed in two stages which include initiating an interserver transaction by inputting the necessary details and secondly authorizing the initiated transaction by authenticating it through an already set method such as PIN or OTP. Kindly follow the sample below as a guide for implementation.

Initiate Interserver Transaction

To initiate an interserver transaction, make a POST request to the following endpoint.

- **Path:** </transaction/direct/initiate>
- **Request Method:** POST
- **Headers:**

Authorization: Provide your public API key here for authentication.
Example: Authorization: 0PUB0629zabkUVDb5rpdM1nGo43F7yOy.

Content-Type: application/JSON

Specifies that the request body is JSON-encoded.

JSON Structure:

Field	Type	Description	Required
amount	Double	The amount to be charged.	Yes
currency	string	The currency for the transaction (e.g., NGN).	Yes
reference	string	Unique reference for the transaction. Which could be alphanumeric	Yes
serviceCode	string	Service code for the transaction.	No
bearer	integer	Indicates who bears the transaction charge. (0 for customer, 1 for business).	No
metadata	object	Custom metadata object.	No
card	object	Card details.	Yes
- pan	string	Partially masked card PAN.	Yes
- cvv	string	Card CVV for authorization.	Yes
- expiryYear	integer	Card expiry year (last two digits).	Yes
- expiryMonth	integer	Card expiry month (1-12).	Yes
customer	object	Customer information.	Yes
- email	string	Customer email address.	Yes
- phoneNumber	string	Customer phone number (with country code).	No
- firstName	string	Customer first name.	No

- lastName	string	Customer last name.	No
callbackUrl	string	URL for callback after transaction processing.	No
narration	string	Description or reference for the transaction.	No
preauthorize	boolean	Indicates if the transaction should be pre-authorized.	No
authorization	object	Authorization details.	Yes
- mode	string	Authorization mode (e.g., PIN, OTP).	Yes
- pin	string	PIN for card authorization.	Conditional (if mode is PIN)
- otp	string	OTP for card authorization.	Conditional (if mode is OTP)
splitConfiguration	array	Array of split configurations (if applicable).	No
pauseSettlement	integer	Indicates whether to pause settlement (0 for no, 1 for yes).	No
pauseSettlementDate	string	Date to resume settlement if paused (use {{pauseSettlementTDate}}).	Conditional (if pauseSettlement is 1)

POST /transaction/direct/initiate

Authorization: 0PUB0629zabkUVD5rpdM1nGo43F7y0y

Content-Type: application/json

Sample Request Body:

```
{
  "amount": 1000,
  "currency": "NGN",
  "reference": "000045mG7mTL022NzzQ0",
  "serviceCode": "0034k1cTL0x2HG",
  "bearer": 0,
  "metadata": {},
  "card": {
    "pan": "50****0580000****99",
    "cvv": "000",
    "expiryYear": 24,
    "expiryMonth": 5
  },
  "customer": {
    "email": "john.wick@gmail.com",
    "phoneNumber": "2348032132100",
    "firstName": "John",
    "lastName": "Wick"
  },
  "callbackUrl": "https://example.com/",
  "narration": "Student Ref number",
  "preauthorize": true,
  "authorization": {
    "mode": "PIN",
    "pin": "0000",
    "otp": "123456"
  },
  "splitConfiguration": [],
  "pauseSettlement": 0,
  "pauseSettlementDate": "{{pauseSettlementDate}}"
}
```

Key Points:

- **reference:** Must be unique
- **pan:** Pass in correct pan details.
- **bearer:** This field could be set to business = 1, customer = 0.

- **currency:** Should be set to NGN.
- **splitConfiguration:** Specifies how the transaction amount should be split.
- **pauseSettlementDate:** Use the current date in the format which must be a future date YYYY-MM-DD.

Authorize Interserver Transaction

To authorize an interserver transaction, make a POST request to the following endpoint. Which requires a transRef and card pin.

JSON Structure

Field	Type	Required	Description
transRef	String	Yes	Unique reference ID for the transaction.
authorization	Object	Yes	Contains authorization-related information.
- mode	string	Yes	Authorization mode (e.g., PIN).
- pin	string	Yes	PIN for card authorization.

- **Path:** [/transaction/direct/authorize](#)
- **Request Method:** POST
- **Headers:**

Authorization: Provide your public API key here for authorization.
Example: Authorization: 0PUB0629zabkUVDb5rpdM1nGo43F7yOy.

Content-Type: application/JSON

Specifies that the request body is JSON-encoded.

POST /transaction/direct/authorize

Authorization: 0PUB0629zabkUVDb5rpdM1nGo43F7y0y

Content-Type: application/json

Sample Request Body:

```
{
  "transRef": "12345ABCDE",
  "authorization": {
    "mode": "PIN",
    "pin": "1234",
  }
}
```

Key Points:

- **transRef:** Must reference an existing transaction in the **Transaction** table.
- **authorization:** Mode can be set to "pin" or "otp".